

Features

- 1) Single codebase for both Android & iOS
- 2) Hot Reload - Instantly see changes in the app w/o restarting
- 3) Fast Performance - Uses dart language & a compiled approach for smooth & high performance
- 4) Open source - Backed by Google & a large developer community, ensuring continuous development

Advantages

- 1) Faster Development - Hot Reload & single codebase reduce development time
- 2) Cost-effective - Since same code runs on both Android & iOS, development cost is reduced
- 3) Reduce performance issues - App runs natively & is free of lags.

Q1b)

- 1) Single vs Separate Codebase -
 - Trad Approach - Separate codebase for Android (Java/Kotlin) & iOS (Swift)
 - Flutter - Single dart based codebase for both platform.
- 2) Rendering Engine vs Native UI components
 - Trad Approach - Relies on platform native UI components which can lead to inconsistencies
 - Flutter - Uses Skia RA to draw everything from scratch.

Why Popular?

- Faster development with Hot Reload.
- Cross Platform Efficiency
- Consistent UI Across Devices
- Improved Performance
- Easy Backend Integration

Q2

- Widget Tree in Flutter

- It is a fundamental structure that represents the UI of the application. A hierarchical arrangement of widgets where each widget defines a component UI.
- Flutter UI is directly entirely built using widgets which can be stateless or stateful. It determines how the UI should be rendered.

• Widget Composition -

refers to building complex UIs by combining smaller, reusable widgets. i.e. Flutter breaks down pages into small blocks that can be reused & nested with each other

Eg

```
class prot extends StatelessWidget {  
  final String name;  
  final String imgUrl;  
  prot({required this.name, required this.imgUrl});  
  @override  
  Widget build(BuildContext context) {  
    return Card(  
      child: Column(  
        FOR EDUCATIONAL USE
```


children [

Image.network(imgUrl)

SizedBox(h: 10)

Text(...)

}

}]

- Benefits of Widget Composition

- Reusability - Can be reused at diff places
- Maintainability - easier to debug
- Performance - rebuilds only necessary ^{part} of widget

1) Structural Widgets

Eg - Material App, Scaffold, Container

Eg - Mtrl App (

name: Scaffold (

AppBar: AppBar (title: Text('Flutter Widget Tree'))

body: Container (

~~padding: Edge~~

),

),

);

2) Interactive Widgets

Eg - TextField, ElevatedButton, GestureDetector

ElevatedButton (

onPressed: () {

print("Button Pressed")

}

);

3) Styling Widgets

Eg - Text, Image, Icon, etc

column (

children [

Text (" Welcome")

Image.network ("https://img1.svc")

];

);

Q3a)

→

- State refers to data that can change during its lifetime of an app -

Eg - Input, UI changes, etc.

- Types of State -

1) Ephemeral

2) Appwide State

• Importance

1) Code Maintainability - & Scalability -

Managing state properly makes the code modular, reusable & Scalable

2) Efficient - UI updates - UI is rebuilt when state changes, efficient state mgmt ensures only necessary parts are updated.

3) Data Consistency & Synchronisation

data remain consistent across different screens & widgets.

11A 11Q

3

- setState - Local state

Pros - Simple & built in, easy to use

Cons - Not scalable

Scenario - When managing simple UI elements within a single widget like toggling dark mode

- Provider - App Wide State

Pros - Lightweight, efficient, flutter-recommended

Cons - Boiler plate code for nested providers

Scenario - When sharing state across multiple widgets like authentication

- Riverpod - AWS State

Pros - eliminates provider limitations, performance ↑

Cons - Requires learning new concepts

Scenario - when building a complex scalable app with global state management:

Q4

a)
→

Step 1 - Create a firebase project

• Firebase console → "Add project"

Step 2 - Register flutter app with Firebase

• FB dashboard → Add App → select platform

Step 3 - Install FB dependencies
Add them in pubspec.yaml
firebase_core, auth,
cloud_firestore
Run - flutter pub get

Step 4 Configure for Android & iOS
- classpath - com.google.gms:google-services 4.3.10
- Add 'com.google.gms:google-services' plugin

Step 5 Initialize Firebase in Flutter

- Benefits -

- Easy to set up & scale
- Provides multiple authentication
- secure file cloud storage
- Firebase Cloud Messaging

245) Flutter Services -

→ 1) Firebase Authentication

Enables secure authentication using email /
pass, Google, Fb, apple, etc.

2) Cloud Firestore

Stores & syncs data in real-time across devices
support structured data, queries & offline access.

Eg -

Firestore.instance.collection('users').add({

});

• Realtime Databases

A real-time JSON based DB that auto-updates the data across devices

Ex -

```
DBReference ref = ..... ("message")
ref.addListener();
```

• Firebase Cloud Messaging

Enables push notifications & messaging between users

Ex -

```
FirebaseMessaging.getInstance().newMessage();
```

• Analytics -

Tracks user interactions & also app performance.

Ex

```
FirebaseAnalytics analytics = ...
analytics.logEvent("... event");
```

• Firebase Hosting -

Deploys & serves web applications securely with automatic SSL.

Data Synchronization in Firebase

FB ensures real-time data synchronisation across multiple devices & platform

1) Cloud Firestore Sync Mechanism

Uses realtime listeners to update UI instantly when data changes

Ex - Firebase Firestore . instance . collectⁿ (users) ...

```
for (var doc in snapshot.docs) {  
    :  
}  
};
```

2) Runtime Database Sync Mechanism

Uses persistent websocket connections for live updates.

Ex Database Reference ref = ...

```
ref.onValue().listen((event) {  
    :  
});
```

3) Offline Data Sync

Firestore caches data locally & syncs changes when the device is online.

Ex Firebase FS . instance . settings = Settings (...);

4) Cloud Functions for Auto updates

Automates backend logic to trigger updates on data changes